

Chapter 3. Demystifying the Medallion Architecture

In [Chapter 1](#), we explored the evolution of Spark and Delta Lake, and introduced you to the Medallion architecture. This design pattern helps organize data logically within modern lakehouse architectures. It utilizes three layers (Bronze, Silver, and Gold) to progressively refine datasets throughout the lifecycles of data ingestion, data transformation, and data loading into various destinations.

Explaining the Medallion architecture to organizations often feels like opening a can of worms, as each layer, meant to address different concerns, lacks clear definitions and descriptive guidelines. This ambiguity leads to more questions than answers, creating a cycle of confusion and inefficiency.

Despite the popularity of this three-layered design, there's significant debate about the scope, purpose, and best practices for each layer. Moreover, the gap between theory and practical application is substantial. In this chapter, I will share insights from my practical experiences on designing each layer of the Medallion architecture by using a theoretical viewpoint. In [Part II](#), the focus shifts from theory to practice. The insights from this chapter are carried forward when engaging in a hands-on exercise where you'll be taught to build a real solution architecture.

The Three-Layered Design

Before discussing the specifics of each layer in the Medallion architecture, it's crucial to understand the high-level purposes and functions of the three primary layers: Bronze, Silver, and Gold. [Figure 3-1](#) illustrates the flow of data from the Bronze layer through the Silver and into the Gold layer, highlighting key processes such as ingestion, processing, and usage for analytics.

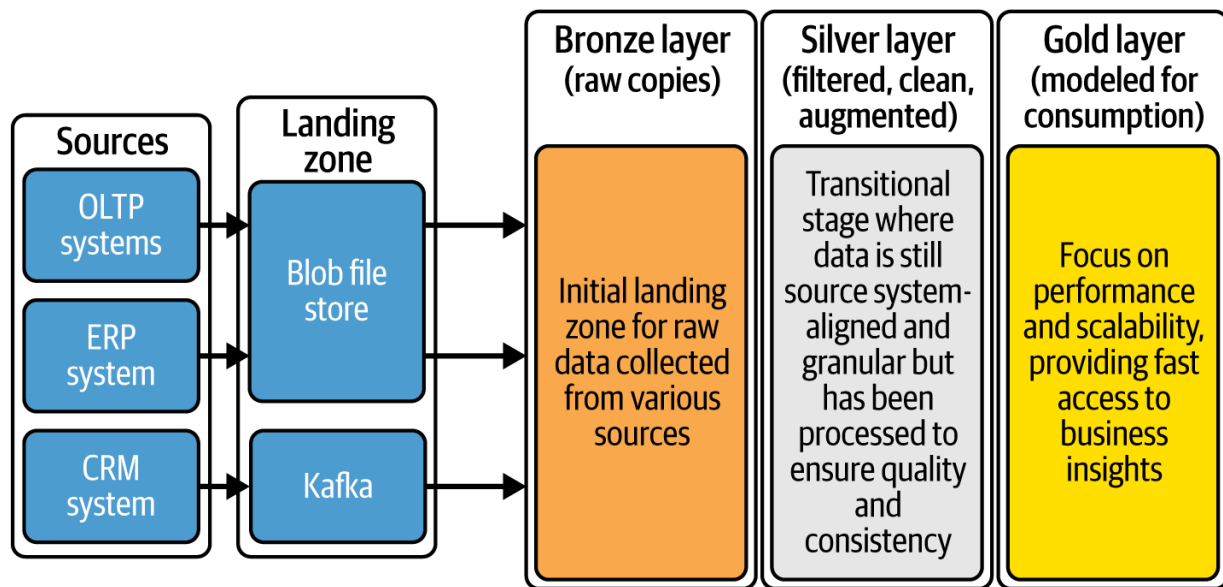


Figure 3-1. The Medallion architecture and its three layers: Bronze, Silver, and Gold

In [Figure 3-1](#), each layer plays a distinct role in the transformation and refinement of data, facilitating a structured progression from raw data collection to data that can be used for data-driven value creation. I introduced the layers in [Chapter 1](#), but this section will provide you with

more details with key characteristics of each layer, setting the stage for a deeper exploration of the technologies and processes involved:

Bronze layer

Acts as the zone for raw data collected from various sources. Data in the Bronze layer is stored in its original structure without any transformation, serving as a historical record and a single source of truth. It ensures data is reliably captured and stored, making it available for further processing.

Its key characteristics are high volume, variety, and veracity. The data is immutable to maintain the integrity of its original state.

Silver layer

Refines, cleanses, and standardizes the raw data, preparing it for more complex operational and analytical tasks. In this layer, data undergoes quality checks, standardization, deduplication, and other enhancements that improve its reliability and usability. The Silver layer acts as a transitional stage where data is still granular but has been processed to ensure quality and consistency.

Its key characteristics are that data in the Silver layer is more structured and query-friendly, making it easier for analysts and data scientists to work with.

Gold layer

Delivers refined data optimized for specific business insights and decision making. The Gold layer involves aggregating, summarizing, and enriching data to support high-level reporting and analytics. This layer focuses on performance, usability, and scalability, providing fast access to key metrics and insights.

Its key characteristics are that it's highly curated and optimized for consumption, so the data in the Gold layer supports strategic business operations and decisions.

NOTE

While the shift from on-premises RDBMSs to decoupled and distributed architectures has transformed technology, the fundamental methodologies of data warehousing and data modeling are still relevant. If you're not confident with these skills, I recommend brushing up on the basics. Great resources include [Kimball Group's books](#) on data warehousing and dimensional modeling, as well as [Star Schema: The Complete Reference by Christopher Adamson \(McGraw Hill Osborne Media\)](#). These great resources provide a solid foundation for understanding the principles of data modeling.

Before we dive into the specifics of each layer, let's start by getting our mindset right about the Medallion layers. It's crucial to think of these layers as logical, not physical. So, when talking about, for example, the Bronze layer, don't frame it as just one physical layer. Instead, view it as a logical layer that could span across several physical layers. This approach will help

us stay flexible and make more sense of the structure. Having said that, let's get started.

Bronze Layer

The Medallion architecture starts with the Bronze layer. The main goal of this layer is to store data from various sources in its original state without any modifications. It ensures that the data is easily accessible for further processing and enables users to explore and analyze it instantly. Essentially, it serves as a queryable reservoir for raw data.

In the next couple of sections, we'll first discuss the processing hierarchy and how this influences the potential Bronze layer's overall design. After that, we'll focus on the archiving and control objectives by discussing processing full and incremental loads, historization, schema management, and technical validation checks. I'll conclude with usage and a summarization the key points about the Bronze layer before moving on to the Silver layer.

Processing Hierarchy

Whether the Bronze layer consists of a single physical layer or multiple sublayers depends on the complexity of the data sources and the organization's needs. In earlier discussions in [Chapter 2](#), we touched on the possibility of having extra landing zones and whether these zones are part

of the Bronze layer. Let's revisit this topic with an example scenario and draw some conclusions.

Traditional source systems often dictate the format in which data must be exported, typically using formats like CSV or JSON. Consider a legacy application that exports data only to a local file storage location. In this scenario, you would integrate the data into your Medallion architecture through batch processing.

Here's a typical process flow: the process begins by staging the data in a landing zone, where you perform minimal processing tasks such as decompressing zipped files, comparing checksums, and generating metadata. Following this, you copy the data to the Bronze layer, converting it into a format like Delta Lake to enhance performance. At this stage, you also carry out basic validations to ensure data integrity. Optionally, you might also classify or encrypt the data to protect sensitive information. Finally, you compare and integrate the data with your existing Bronze datasets to maintain consistency and completeness. We'll revisit these details when implementing the Bronze layer in [Chapter 5](#) using external tables or Delta copies.

NOTE

Encrypting the raw data is essential to protect personally identifiable information (PII) from unauthorized access. Therefore, it's not uncommon to encrypt the data before it lands in the Bronze layer. This encryption can be done using open source encryption frameworks, such as [Fernet](#), or other encryption tools.

In the data processing hierarchy, the Bronze layer often includes multiple sublayers: an internal staging area, a processing and refinement area, and eventually storage in Delta format. Thus, whether the Bronze layer is a single physical layer or multiple sublayers depends on data source complexity, organizational requirements, and the decision on whether to include the landing zone as part of the Bronze layer.

With the processing hierarchy established, it's crucial to focus on how data is processed and managed over time because each data source is unique and the way data is delivered can vary significantly between source systems. Some systems provide full extracts, others deliver delta increments, and still others stream data continuously. Therefore, deciding on the appropriate processing pattern is essential. Let's explore these strategies in detail, starting with processing full data loads, followed by processing incremental loads.

Processing Full Data Loads

Full data loads involve transferring the entire dataset from the source system, rather than just changes, ensuring all data is available for downstream processing. When processing full data loads, large batches of data are typically handled at fixed intervals, often with the optional use of landing zones. After validation, the data is stored in its raw form and accumulated in folders alongside all previous data deliveries. At this stage, minimal transformations are usually performed; sometimes data is filtered, or sensitive columns are encrypted. After accumulating the data, the most recent delivery is typically either converted to the Delta Lake format or typically exposed as external tables for processing in the next layer. We will revisit these patterns in [Chapter 5](#), where we explore the processing of full data loads in greater detail and demonstrate concrete examples.

Processing Incremental Data Loads

The pattern of loading increments, or “delta loads,”¹ is often employed in scenarios where only changes or updates to the data need to be processed, allowing for more efficient data handling and reduced processing time compared to full data loads. In this approach, you typically receive only the data that has changed, often in the form of events, for example. This data is usually staged in a preliminary layer, where you might first apply some lightweight transformations. These could include changing the file format

(e.g., from JSON or Parquet) and converting the data to the Delta Lake format or making simple data type corrections. After these initial steps, you can utilize [append mode](#) or [merge operations](#) to integrate this incoming new data with the existing data already present in the Bronze layer. Let's explore these methods further:

Append mode

In the context of Delta Lake, “append mode” means that new data is added to an existing Delta table without modifying or deleting the existing data. This is useful for scenarios where you want to continuously add new records to a dataset, such as logging or streaming data, without affecting the existing data.

Here is an example where `df` represents a DataFrame, a structured collection of data with named columns. Using Spark, you can read data into the DataFrame and subsequently append it to a Delta table:

```
df = spark.read.json(f"{filePath}/events/*.j  
df.write.format("delta").mode("append").save
```

Append mode does not address updates or deletions—it simply appends new data to the end of the dataset. Merge operations are more sophisticated than a simple append because they allow for both

the insertion of new records and the updating of existing records based on some key.

Merge mode

In Delta Lake, the “merge mode” provides a way to perform upserts (a combination of updates and inserts) to a Delta table. This is particularly useful when you need to update existing records and insert new ones in a single operation, based on a specified condition.

Suppose you manage a table containing customer information, including a status flag labeled `is_current` for customers present in the source system within the last 180 days. You can use a `MERGE` operation on the Delta table, which efficiently allows for multiple changes simultaneously:

- Insert new customers.
- Update the status of customers who have recently returned.
- Modify the status of existing customers in the Delta table who are now inactive.

Delta Lake efficiently handles these operations behind the scenes, ensuring data integrity and streamlined processing. We'll connect back to this merge

approach in [Chapter 5](#).

Processing incremental loads is an efficient method to manage large datasets by processing only the changes since the last load instead of reprocessing the entire datasets. In scenarios with large source system tables, such as those containing millions of rows, this approach could be particularly useful. Furthermore, incremental loading is also essential for efficiently updating data in any layers, such as the Silver and Gold layers, without reprocessing the entire dataset. A consideration for incremental loading is to combine it with a [change data feed](#) to push incremental changes to your next layers.

TIP

By implementing incremental processing with Delta Lake and Spark, and utilizing configurations like `trigger(AvailableNow=True)`, organizations can manage their data more effectively and economically.

For incremental loading to work effectively, your (source system) tables need to have unique incremental identifiers or `updated_at` columns. These markers allow you to detect any new or updated records since the last load. Furthermore, it's crucial to ensure that the source system does not update records that are older than the last fetched increment. If updates on older records are possible, standard incremental loading might miss these changes. In such cases, using CDC or similar tools might be necessary.

These tools are designed to capture all data changes—inserts, updates, deletions—from the transaction logs and can be more reliable for ensuring complete data synchronization.

For streaming, you would typically specify the initial position's startingVersion by using the Delta (or Iceberg) transaction log. This allows you to pick up exactly where you left off without needing to run any additional commands.

Alternatively, you could use a max function on the identifier or *updated_at* column from your Bronze layer to determine the most recent entry processed. Once you have the highest value (`max(updated_at)`) from the Bronze layer, you can then query the source data for records with identifiers or timestamps greater than this value. This strategy ensures that only new or updated records since the last load are processed.

Maintaining a metadata control table that tracks processed records is another recommendation. This helps manage data flow and ensures consistency, particularly in complex systems.

For a deeper understanding of incremental delta loads and their practical usage, consider watching “Incremental Refresh with Your Warehouse Without a Date in Microsoft Fabric”. This video on incremental delta loads provides comprehensive insights and examples to help you implement this technique effectively.

By capturing incoming data in its incremental form, you can integrate it with the complete historical dataset. This process of historization is crucial not only for building a comprehensive archive but also for ensuring that current data is readily accessible for further downstream processing. We'll explore this topic next.

Data Historization Within the Bronze Layer

The Bronze layer is designed primarily for storing complete snapshots or copies of data, similar to what you'd find in a staging environment in traditional data warehouse environments. This layer is especially useful if your method involves fully extracting and overwriting data, as it allows you to maintain a comprehensive history of all data deliveries. The Bronze layer usually also serves as a robust archive, preserving data for many years, which is vital for audit purposes. However, there may be instances where you need to clean up data that has been processed and is no longer required.

In a Medallion architecture, archiving data is straightforward when you use an efficient storage format like Parquet or Delta and organize it in folders.² By structuring your data in interval-partitioned tables and sorting them into folders labeled with a YYYY/MM/DD or datetime format, you keep your data well-organized and manageable. This systematic organization not only keeps your data tidy but also simplifies access and auditing. If you ever need to look back at data from a specific day, you can easily recreate its state, ensuring you can retrieve historical data accurately whenever needed.

However, it's important to note that the Bronze layer is not intended for historization of data in the sense of representing a fully processed SCD2 table. Data in the Bronze layer is often considered immutable, meaning it is read-only. While you may append or merge data to incrementally grow the dataset over time, updates or processing of the data are not typical practices at this stage.

NOTE

You can apply data versioning by using the [Delta Lake time travel feature](#). It is primarily designed for data recovery, auditing, and reproducing experiments or reports, rather than for comprehensive long-term data archiving.

Historization is crucial, especially when dealing with Bronze-level datasets. This process enables the archiving of data iterations over time, which can be essential for auditing or debugging purposes. Furthermore, historization supports the ability to perform reloads and recoveries. This feature is vital for handling instances of poor-quality data or disruptive changes in schemas, allowing you to revert to a previous version of the dataset and reprocess it.

While delving into the complexities of processing historical data, you encounter a significant challenge: schema evolution. As businesses grow and data sources are frequently updated, the database structure (schema) can change. This evolution can lead to inconsistencies in data processing,

making it essential to manage schema changes effectively. We'll explore this topic in the next section.

Schema Evolution and Management

The Bronze layer is crucial for managing widely varying and ever-changing data schemas as data enters the system. Effectively handling schema evolution at this stage sets the foundation for all the data processing and analytics that follow. You can tackle incoming schema changes either in the Bronze layer or wait until the data moves to the Silver layer. The timing of addressing these changes depends largely on your specific needs and the characteristics of your data.

When managing schemas, organizations face design choices, each with its own approach to schema evolution:

Schema-on-read

This method applies the schema dynamically as you read the data. With this, data is stored without a strict schema in place. The schema is only inferred or applied when the data is accessed during processing (reading). This flexible approach requires the system to recognize and possibly adapt to various data schemas right after the point of ingestion. This approach is particularly useful for semi-structured or unstructured data or structured data without any built-in schema enforcement, such as CSV or Parquet files.

Schema-on-write

This approach involves defining the schema—like the table and column names, the data types, and the primary keys—as you write the data into storage. With this method, you must define the schema up front, making sure that the data conforms to this schema right from the start, before writing. It's a more rigid approach compared to schema-on-read, as it enforces schema consistency from the beginning. When applied, it's generally used for structured data.

When using Delta Lake, if there is no predefined schema, the system will establish an initial schema by using the `StructType` from the `DataFrame` that is being converted to Delta format. In Apache Spark, `StructType` refers to a container that defines the structure and types of columns in a `DataFrame`.

In the Bronze layer, schema-on-read is a common approach where data is stored in its original format without enforcing a schema during ingestion. This method offers flexibility since it does not demand an up-front schema definition and can gracefully handle changes in data structure. For instance, imagine you partition your data daily using a YYYY/MM/DD time partitioning format, storing each day's data as Parquet files within corresponding folders labeled by date. With schema-on-read, you can immediately access and read this data across different days without needing to predefine how the data structure should look. This approach allows for

easy integration and analysis of data collected over time. However, there are several important factors to consider.

Firstly, relying solely on schema-on-read does not allow for blind data processing. You may need to detect and log schema changes, handle failures, restore data using Delta Lake's time travel feature, and add schema-related metadata for more refined processing in later stages. We'll delve into these aspects and introduce tools like Auto Loader in the detailed discussion in ["Databricks Auto Loader"](#).

Secondly, defining a data update method is essential, especially for historization and integration of data. Options include continuously appending to or merging with existing datasets, completely overwriting them, or applying time-based partitioning. The choice largely depends on the nature of the data, its usage, the specific system requirements, and performance considerations. For instance, appending is commonly used for transactional data because this type of data grows incrementally with each new transaction. Overwriting might be chosen for datasets that receive regular source corrections, or where only the most recent data snapshot is relevant. Time-based partitioning can be implemented to enhance query performance and manage large datasets effectively, especially when data access is time-sensitive.³

Therefore, the Bronze layer often combines schema-on-read and schema-on-write techniques. Initially, schema-on-read is applied in the landing or

pre-Bronze layer. Hence, you first read data without explicitly declaring the data schema. Then, as you move the data to a layer where it becomes “queryable,” you switch to schema-on-write. For this critical layer, it’s wise to choose a storage format like Delta Lake that supports schema evolution. Delta Lake allows the data schema to adapt as it evolves, eliminating the need to overhaul or reload the entire dataset. This capability keeps the data current and usable, even as changes occur. We’ll revisit this feature shortly.

However, be mindful that keeping schemas consistent can pose challenges, especially if the source systems are prone to frequent changes. This requires vigilance and possibly more sophisticated schema management strategies to ensure data integrity and system reliability. For this reason, to keep things running smoothly, it’s essential to maintain a schema that consistently matches the source system as data enters this layer. You usually set up the schema during the data ingestion process to comply with specific standards, either through data definition languages (DDLs) or programmatically using tools like Spark SQL or Databricks Auto Loader. We’ll revisit this subject in more detail in [Part II](#).

MergeSchema and Schema Enforcement

Delta Lake offers a set of features for managing data schemas, such as handling schema evolution and schema enforcement. The features you apply depend on your system’s specific needs and design, and you can see them across all layers of the Medallion architecture.

For example, as data evolves, there might be a need to modify the table schema to accommodate new types of data (e.g., adding new columns). `mergeSchema` simplifies this process. It's a feature designed to help manage and evolve the schema of Delta Lake tables over time. To enable this, add the following option in a write operation in Apache Spark:

`.option("mergeSchema", "true")`. Delta Lake then takes care of adjusting the table schema in the following ways:

- If a column exists in the source DataFrame but not in the Delta table, Delta adds the new column. All existing rows will have a null value in this new column.
- If a column is in the Delta table but not in the source DataFrame, it remains unchanged. New records will have null values for these missing columns.
- Adding a `NullType` column sets all existing rows to null for that column.
- If a column with the same name but a different data type exists, Delta Lake tries to convert the data to the new type.⁴ If the conversion fails, Delta Lake throws an error.

In case of errors or failures, you can roll back the table to a previous version using Delta Lake's table history. This rollback feature allows you to recover from schema evolution issues. After reverting, you can reprocess the corrected data using the appropriate schema. For more detailed information

on how Delta Lake supports schema evolution, check out the blog posts at [Delta Lake Schema Enforcement](#) and [Delta Lake Schema Evolution](#).

To handle schema changes more strictly, you can use the [Delta constraints](#) feature, which is closely aligned with the schema-on-write approach. This method ensures that the data written to a Delta table strictly matches the table's schema, and it rejects any writes that do not conform. As data enters the system, Delta Lake verifies its adherence to the predefined schema. Any deviation results in an error, thereby maintaining the integrity and consistency of your data. However, this approach can make it challenging to handle persistent and disruptive changes in the source system's schema.

If you encounter changes that affect compatibility, for example, changes that cannot be handled via the `mergeSchema` option, you'll need to reconcile the schemas. When adjusting a Delta table's schema, you have several options:

Using `SQL ALTER COLUMN` statements

Updating schemas with `SQL ALTER COLUMN` statements involves making direct changes, which requires careful coordination between the source system owners and the data platform managers. One effective way to manage these modifications is by checking the operations or scripts into a repository. This method is very precise but can be labor-intensive and prone to errors due to the intricate coordination required.

Automated schema evolution

You can implement automated tools or scripts that can detect schema changes in the source system and automatically generate and apply the necessary `ALTER` statements to the target system. We'll discuss this approach in more detail in [“Handling Schema Evolution”](#).

Using a metadata-driven framework

This recommended approach not only helps maintain the original and new target schemas but also provides a robust way to manage changes systematically. By maintaining mappings between schemas, you create a scalable method that can handle complex transformations and schema evolution without extensive manual intervention. This ensures the metadata repository is always up-to-date and accurately reflects the schemas in use. It might be helpful to integrate this with your continuous integration and deployment (CI/CD) pipeline to automate updates and deployments. We'll connect back to this approach in [Chapter 5](#).

Restricting disruptive changes

Sometimes, you might consider requiring source system teams to avoid making disruptive changes. Thus, you'd only allow backward-compatible changes to be made. While this places constraints on the source systems, it can significantly reduce the complexity and frequency of schema updates.

Alternatively, for very disruptive source system schema changes that cannot be reconciled with the previous version, you might consider creating a new version of the data pipeline. In this scenario, the new data would land in a new target location, allowing you to manage significant schema changes more effectively. This allows you to maintain multiple versions of the schema concurrently, and applications or processes can specify which version of the schema they need to interact with.

As we explore schema evolution and the strategies to adapt data structures to evolving business needs, it becomes evident that one of the essential aspects of managing these schema changes is the necessity to implement robust technical validation checks to detect errors and unknown changes. These processes are closely interlinked and, together, they significantly improve data management.

Technical Validation Checks

Technical validation checks are crucial, as issues with data integrity, accuracy, completeness, and consistency can have significant operational and strategic impacts on any organization. Within the lakehouse architecture, the Bronze layer acts as the initial repository for raw, typically unstructured data. Here, data is stored in its native format, which may include inconsistencies or errors.

Implementing validations, such as format, schema, and completeness checks, at this stage is essential for several reasons. First, it ensures that the data conforms to specified standards, which helps maintain consistency across the dataset. This is particularly important as data from various sources may not adhere to a stable structure. Moreover, early validation helps in identifying and addressing data integrity issues. In particular, it's useful when applying a schema-on-write approach. By catching errors at the onset, it prevents the propagation of inaccuracies to subsequent layers—Silver and Gold—where data is further processed and refined for analytical purposes.

The design of the Bronze layer centers on strong data validation controls because it acts as the primary shield for technical validation checks and observability. You have different options: perform these checks in one go, break them into segments, or integrate them during data transfer to the Silver layer. To validate data effectively, many companies use scripts and keep a metadata repository. This repository holds technical schema details from their source systems, enabling automatic enforcement of data validation rules, both declaratively and dynamically.

The design of your Bronze layer also hinges on your tolerance for data integrity issues. If you anticipated a failure in downstream processes or if data consumers cannot handle these issues, you should halt the pipeline. This approach, known as *intrusive data integrity processing*, involves storing validated and incorrect data in a separate folder or table. By doing

this, you can easily identify and rectify the issues, ensuring these datasets do not enter your actual Bronze layer.

Conversely, if you can tolerate data integrity or completeness issues, you may opt to continue processing the data, a method referred to as *nonintrusive data quality processing*. In this scenario, both validated and incorrect data are stored directly in the target Bronze-layer folder or table. This allows you to proceed with data processing, but be mindful that you may need to address these issues in another layer (typically Silver) later on.

Most of the checks validate the technical aspects of the data source to prevent data from advancing to the next zone unless it meets the set of predefined standards. If data doesn't meet the next zone's standards, it's the responsibility of the data owners to rectify this in close collaboration with application teams and data engineers. These team members, who manage and source the data, must ensure that the data fulfills the integrity requirements stipulated by the consumers or users of the lakehouse.

In this way, the Bronze layer not only serves as a repository for raw data but also as a critical checkpoint for data integrity and accuracy, ensuring only data that meets predefined quality standards progresses further into the lakehouse architecture. This proactive approach to data management helps safeguard the integrity and usability of the data throughout its lifecycle in the lakehouse.

A variety of tools are available for managing data ingestion, validation, and orchestration processes, and ensuring that your data's integrity is intact:

- You can use Delta Lake's schema validation features for addressing compatibility and consistency issues. For instance, schema enforcement ensures that developers adhere to predefined standards, keeping the tables clean. When data is written to a table, Delta Lake performs schema validation to check if the data's schema matches the table's schema. If the schema is compatible, the validation passes, and the write succeeds. However, if the schema is incompatible, Delta Lake cancels the transaction, preventing any data from being written.
- You can use data quality frameworks or tools, such as Delta Live Tables, Great Expectations, data build tool (dbt), Ataccama, or Monte Carlo Data. We'll come back the differences between these tools in [Chapter 6](#).
- You can use custom solutions with scripts, code, and schema metadata to validate data integrity. This approach is often used when organizations have specific requirements that are not met by existing tools or frameworks. Custom solutions can be tailored to the organization's unique needs, providing a more flexible and customizable way of working.
- Azure Data Factory supports [schema drift detection](#), which makes you less vulnerable to changes in upstream data sources.

Now that we've covered some design considerations, let's explore how the Bronze layer is used and its role in the broader context of data utilization.

This will help us understand its importance and integration within the larger data management framework.

Usage and Governance

Some data practitioners argue that business users can benefit from querying or conducting ad hoc analyses on Bronze data. However, raw data poses significant challenges because it demands a deep understanding of the source system's design and the intricate business logic embedded within. The frequent presence of numerous small tables complicates usability. Being tightly coupled to the original source system presents significant challenges, especially when the source system undergoes changes. These complexities often discourage direct use of Bronze-layer data for business analysis.

To ensure proper governance, implement strict access controls to prevent unauthorized data access, manipulation, or deletion. The data in the Bronze layer is immutable, meaning it remains unaltered from its original state, although exceptions exist when handling overly technical data. This immutability requires maintaining detailed logs that record data ingestion times, sources, and any system interactions to enable accurate traceability.

It's crucial to set up alerts to monitor any anomalies in data size, format, or arrival times to maintain the integrity of data pipelines. Developing and regularly updating an incident response plan is vital for addressing issues

related to incorrect data deliveries effectively. Implementing best practices, such as thoroughly documenting changes and maintaining a clear reference catalog, is also recommended.

In essence, the Bronze layer serves as a foundational staging area in the Medallion architecture. It collects raw data from diverse sources and processes it by validating and converting it into the preferred format, such as Parquet or Delta. Although data in this layer is typically immutable, there are exceptions, particularly when it involves enriching the data with metadata or applying filters and transformations to manage sensitive information effectively. Data in this layer may either be completely raw or slightly augmented.

The Bronze Layer in Practice

The Bronze layer serves as the foundation for capturing and storing raw data in its most authentic form. It functions as the base input layer that reliably ingests and preserves all source data in an unaltered state, safeguarding against data loss and corruption. This makes it a reliable foundation for (re)loading and further processing.

The Bronze data layer should not be viewed as rigid. It should be tailored to meet the specific needs of your organization. This could involve adding extra landing areas, preprocessing layers to handle incremental data deliveries, and data validation parking zones, as illustrated in [Figure 3-2](#).

Or, you can treat (parts of) the Bronze layer as a virtual layer, where data is not physically stored but is instead represented by a logical representation of the source systems. By adapting the design to your organization's needs, you can maximize the effectiveness of the Bronze layer.

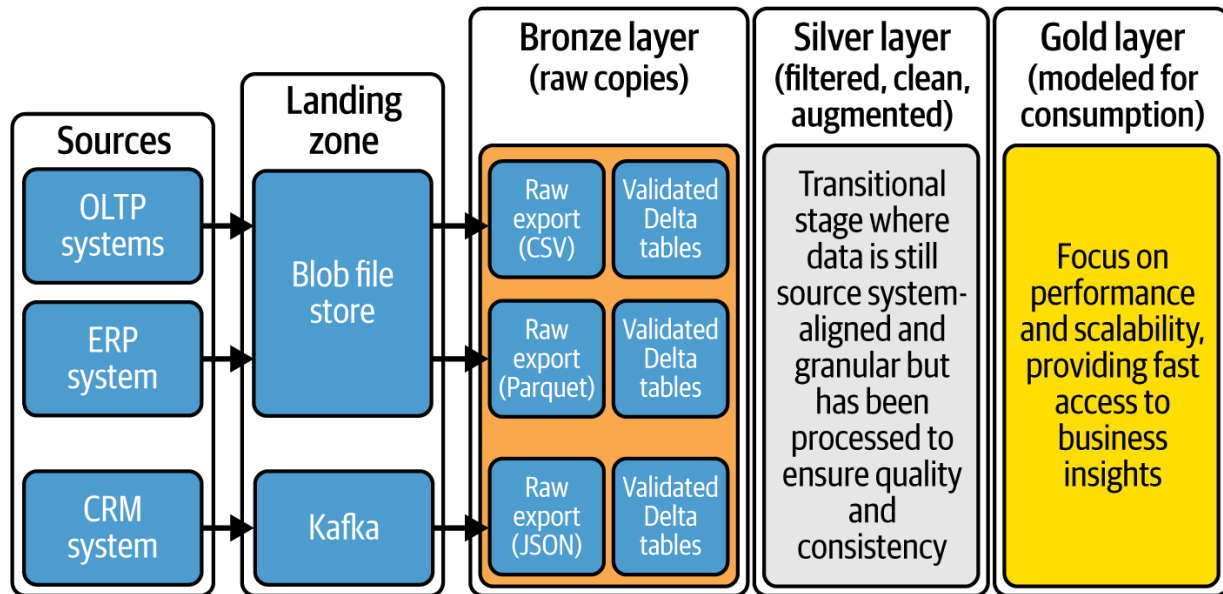


Figure 3-2. How the Bronze layer could look in practice

While the Bronze layer is indispensable for data integrity and historical accuracy, the rawness of the data limits its direct applicability for analytical purposes. Data in this state often contains inconsistencies, redundancies, and anomalies that can skew insights if not addressed. Furthermore, the Bronze layer is tightly coupled with source systems, which introduces risks and dependencies. Thus, transitioning to the Silver layer becomes essential.

Silver Layer

Having validated the data in our Bronze layer, where it now resides in a queryable state, we can progress to the Silver layer. This layer focuses on cleansing and enhancing the data. Here, formats are standardized for elements such as date and time, reference data is enforced, naming standards are conformed, duplicates are removed, and a series of functional data quality checks are conducted. Additionally, low-quality data rows are discarded and irrelevant data is filtered out. Most importantly, at this stage, data generally isn't yet merged or integrated with data from other sources.

Designing a Silver layer, similar to the Bronze layer, requires making complex decisions. Let's dive into these nuances and explore some common trade-offs. We'll start with data cleaning activities, then tackle a design consideration. Next, we'll discuss its usage, which includes operational querying and machine learning applications.

Cleaning Data Activities

Frequently, organizations encounter poor-quality source data, necessitating thorough cleansing. Many data cleansing tasks can be automated within the loading and processing phases, although some are more effectively addressed at the operational source. You want to avoid that these data quality issues pop up elsewhere. Source systems, for example, could also

have interfaces to other applications. Therefore, it's recommended to fix data quality issues at the source of origination.

The data cleansing process can be governed by specific ETL rules or, in some cases, through mapping tables. It's also important to recognize that developing a cleaning process is complex and often requires multiple revisions. After loading data into the Gold layer, you may discover issues that necessitate a return to the Silver layer, and sometimes even to the Bronze layer, to resolve these issues effectively.

The following activities are examples of aspects of data cleaning:

Reducing noise and removing inauthentic data

Eliminate irrelevant data, such as unnecessary columns or rows, to enhance data quality and reduce storage requirements. This step also involves removing inauthentic data, which does not accurately reflect the true (golden) source.

Handling missing values

Assess missing data within your dataset and decide the approach for addressing it—options include removal, substitution with a default value, or employing imputation techniques to estimate missing values based on surrounding data.

Removing duplicates

Verify that your dataset is free of duplicate records unless specifically required (e.g., for retention compliance). Duplicates can distort analysis and lead to inaccurate models.

Trimming spaces

Remove unnecessary spaces in data entries, particularly in string data types where leading and trailing spaces could impact sorting, searching, and other string manipulation tasks.

Error corrections

Identify and rectify errors in data entry, such as typos, incorrect capitalization, or erroneous units. This category also covers the detection and correction of outliers—data points that deviate markedly from the norm.

Consistency checks

Confirm data consistency throughout your dataset. This includes standardizing abbreviations, terminologies, and units of measure (e.g., consistently using *NL* instead of alternating with *NETHERLANDS*).

Standardizing formats

Ensure date formats are consistent (e.g., using [YYYY]-[MM]-[DD]), recognizing that different locales may prefer alternative

formats (like [DD]-[MM]-[YYYY]).

Correcting types

Ensure that columns are assigned correct data types, such as numeric, date, or floats, as appropriate.

Fixing ranges

Check that data in each column adheres to specified value ranges.

Fixing uniqueness

Ensure that data in each column maintains uniqueness, where required.

Fixing constraints

Validate that no column is orphaned; each child record must correspond to an existing parent record in the parent table.

Masking sensitive data

Conceal any PII appearing in clear text prior to data usage to protect privacy and comply with regulations.

Anomaly detection

Detect and fix anomalies that could indicate data quality issues. For instance, a sudden spike in sales data could be a sign of a data quality issue.

Applying master data management

Process data to ensure the accuracy, uniformity, and consistency of an organization's shared critical data.

Standardizing data

Process data like addresses, phone numbers, locations, and reference codes to ensure consistency and accuracy across systems.

Conforming data

Conform data using a common data model to standardize and harmonize information across different systems.

This list is not meant to be exhaustive. Writing cleaning rules is a complex task that requires a deep understanding of the data and the business processes that generated the data. Remember, after cleaning, the table structures are generally the same as in the Bronze layer. It's important to note that incorrect or rejected data isn't typically deleted. Instead, it's flagged or filtered out and then stored in a sibling quarantine table within the Silver layer. We'll revisit this topic in more detail with code snippets and examples in [Chapter 6](#).

Now that you have cleaned the data, what should you do next? Should you retain it in its original structure, or should you remodel it? Let's explore these questions in the following sections.

Designing the Silver Layer's Data Model

The design of the Silver layer's data model is a critical aspect of the Medallion architecture. This is often a hot topic, as there are many possibilities. It is significantly influenced by the number of data sources collected, the extent to which these sources share the same key elements or objects that can be matched, related, and combined across different source systems, and the need to harmonize these overlapping key elements for an integrated perspective.

We'll start with conforming and renaming columns, followed by denormalization. Next, we'll discuss SCDs, the use of surrogate keys, and harmonization with other sources. Finally, we'll discuss 3NF and data vault modeling techniques and close with a discussion on the Silver layer's usage and governance.

Conforming and renaming columns

In a Medallion architecture, it's common for the Silver and Bronze layers to align (straight one-to-one) in terms of their tables, though the way data is presented in these layers might slightly differ.

While it may not be a standard practice everywhere, many organizations I've worked with consider renaming columns to apply consistent naming conventions a best practice in the Silver layer, and I agree with this approach. By renaming and commenting on columns, data teams ensure

these names are descriptive and truly reflective of the data they represent.

This simplifies both the navigation and manipulation of datasets.

Additionally, it enhances communication among team members who rely on the same data, thereby minimizing confusion and reducing errors during data processing. Here is an example of how to use SQL to create a table and rename columns with technical names to more user-friendly names in a query:

```
/** Create table with user-friendly comments */  
CREATE TABLE silver.customer (  
    CustomerID BIGINT COMMENT 'Customer Identifier',  
    FullName STRING COMMENT 'Customer Full Name',  
    Region STRING COMMENT 'Region Code',  
    SignupDate DATE COMMENT 'First Sign Up Date',  
    LastLogin DATE COMMENT 'Last Login Date'  
);  
  
/** Table and field names are easy to read */  
INSERT INTO silver.customer  
(CustomerID, FullName, Region, SignupDate, LastLogin)  
SELECT  
    custID AS CustomerID,  
    CONCAT(f_name, ' ', l_name) AS FullName,  
    rgcd AS Region,  
    sigdt AS SignupDate,  
    lldt AS LastLogin
```

```
FROM  
    bronze.cust_data;
```

Renaming columns often includes standardizing and conforming data within tables to ensure consistency and reliability. For example, you can adjust data to fit within a specific range or standardizing categorical data by using consistent codes or labels. These type of standardizations in the Silver layer support more seamless transformation and integration of data for the the Gold layer, where data is refined for specific business insights.

Using appropriate data types prevents implicit type conversions, which can slow down queries by consuming extra computational resources. By selecting the smallest data type that can accommodate your data, you can also achieve benefits in terms of data storage efficiency.

NOTE

Applying specific ranges or standardizing categorical data within Silver tables is an activity that can be closely related to master data management (MDM). MDM focuses on ensuring that an organization's shared data—often called master data—is consistent and accurate. Standardizing data within the Silver tables aligns with these objectives, as it enhances data uniformity and reliability across the organization. We'll revisit MDM in more detail in [“Master Data Management”](#).

While it's is closely related to the Bronze layer, the practices of renaming columns, establishing uniform data types, and ensuring consistent data that

take place in the Silver layer might be particularly effective for later transformation.

We've just seen how the Silver layer follows the structures in the Bronze layer. Sometimes the alignment between the Silver and Bronze layer tables isn't as tight. Deciding how strictly to align these requires considering some trade-offs. In the next section, we'll explore denormalization, which illustrates how the alignment between the Bronze and Silver layers can become less precise.

Denormalization

To optimize query performance in data modeling, you can consolidate data into fewer tables, and, by reducing the need for complex joins, queries can run faster. This process is also referred to as *denormalization*. It sometimes occurs in the Silver layer and is even more common in the Gold layer. If you expect to frequently reload and intensively read data in your Silver layer, it's best to use a more denormalized data model as it generally offers better performance.

With a denormalized model, data is organized around commonly queried subject areas, eliminating the need for extensive joins and aligning well with distributed and column-based storage architectures. This approach simplifies the data structure, making it easier to access the required

information quickly and improving query efficiency. Here is an example of how to denormalize data using a SQL statement:

```
INSERT INTO silver.customer_return
SELECT
    cus.CustomerID,
    cus.FullName,
    c.Region,
    c.SignupDate,
    c.LastLogin,
    st.OrderID,
    st.OrderDate,
    st.OrderAmount
FROM
    silver.customers cus
INNER JOIN
    silver.orders st
    ON cus.OrderID = st.OrderID
INNER JOIN
    silver.customer_details c
    ON cus.CustomerID = c.CustomerID
```

When applying denormalization, table volumes start to increase because redundant data is intentionally added to improve query performance. This redundancy results in more data being stored in the table, leading to a increased table volumes, which might impact performance depending on

the scenario. In light of this, many organizations manage their tables via [maintenance and optimization jobs](#).

Slowly changing dimensions

To build a comprehensive historical record of all changes over time, it's necessary to reprocess data into what are known as slowly changing dimensions type 2 (SCD2). This involves changing tables by adding additional columns such as `start_date`, `end_date`, and `is_current` to track changes more effectively. However, this process is complex and challenging, as defining a “change” can vary significantly between the source and target tables. Creating business keys or generating hashes are crucial in this process, as they help in making comparisons that determine how to handle the source data.

A business key

In data modeling and database design, a business key refers to an attribute or set of attributes that can be used to uniquely identify a business concept or entity. Business keys are also often referred to as natural keys. Examples include CustomerID as a primary key in a customer relationship management (CRM) system or a Social Security number.

A hash

In data handling, a hash refers to a fixed-size result generated from input data of arbitrary size, using a hashing algorithm. This unique result, or “hash value,” serves as a digital fingerprint for data.

Keep in mind that the structure of the target tables impacts this process; any columns not included in the target should not be part of the comparison.

Within the data engineering community, there is a debate about whether building SCDs should take place in the Silver layer or be reserved for the Gold layer. Most engineers argue that the Gold layer is more appropriate for SCD2,⁵ as these tables are expensive to create and store and are generally underused. Thus, Silver tables should mainly contain current records.

However, this perspective has its nuances. If your Gold layer involves consolidating and merging various data sources, there might be value in creating a historical perspective in the Silver layer, especially when the authentic context from the source is crucial. This is often the case when building machine learning models that depend on historical data in its original context. Additionally, for operational reporting, maintaining historical data in its original form could eliminate the need for an operational data store close to the source. This is increasingly applicable where organizations have decreasing access to authentic data in operational systems, such as SaaS, outsourced services, and NoSQL solutions, and so on. Ultimately, deciding whether or not to implement SCDs in the Silver layer involves nuances and depends on specific project requirements.

Surrogate keys

The debate about SCDs relates to the one about whether or not to include surrogate keys in the Silver layer. A surrogate key is a unique identifier assigned to records, which has no inherent business meaning but serves to uniquely identify a record within a table. Surrogate keys are typically generated using the auto increment feature, or alternatively, by hashing or concatenating multiple columns to create a unique identifier. Their primary advantage is their stability and permanence—they never change. A surrogate key is helpful for tracking changes in dimension attributes over time because its value doesn't change even if the business key changes.

My perspective is that surrogate keys do not typically belong in the Silver layer. The subsequent stage, where data from different sources is combined and merged to build dimensional and fact tables, is where surrogate keys should first be created. At this stage, tables are joined using natural/business keys to look up and add surrogate keys. Therefore, the Silver layer should focus on cleaning and better representing the dataset, rather than creating surrogate keys.

However, if there is a strong preference for using surrogate keys in all SCDs, it's possible to implement them in both the Silver and Gold layers. In this scenario, the surrogate key generated in the Silver layer could be used as a lookup key to find the corresponding surrogate key in the Gold layer.

This approach can work, but it requires careful implementation to ensure data consistency and integrity across layers.

We have approached the end of discussing the Silver's design. In the Medallion architecture, as you learned, the Silver and Bronze layers share a close alignment. However, there are slight differences in how data is presented within these layers. For instance, columns may be renamed, and data may undergo standardization. Additionally, the underlying storage is optimized to enhance performance. Despite these modifications, the data typically maintains its source-oriented nature. At this stage, it is not (yet) integrated with data from other sources. Let's discuss this aspect, as well the need for enrichments.

Harmonization with Other Sources

I often address questions about whether sources within the Silver layer should already be integrated across sources so that an integrated or enterprise view can be created. This type of activity is nuanced.

Generally, my advice is to keep things separate for easier management and clearer isolation of concerns. To facilitate this, I recommend you don't merge or integrate data from different source systems prematurely. Doing so can create unnecessary coupled connections between applications. For example, a user interested in data from a single source might inadvertently be linked to other systems if the data is combined immediately after being

loaded into the Bronze layer. As a result, these users could potentially experience impacts from other systems.

Thus, if your goal is to maintain an isolated design, it's better to move the integration or combination of data from different sources to the Gold layer. This strategy aligns with maintaining clear boundaries and minimizing dependencies between different systems. This philosophy also applies to aligning your lakehouses with the source-system side of your architecture. Therefore, if you are building (source system-aligned) data products and are keen on aligning data ownership, I caution engineers against prematurely cross-joining data from applications across different domains.

However, I've also seen organizations that prefer to integrate data from different sources in the Silver layer. In this scenario, the Silver layer acts more as traditional data integration layer, enabling the data to be combined and harmonized across entities before moving it to the Gold layer for final consumption. In this approach, I generally see more traditional data modeling techniques being used, such as the 3NF or data vault. Let's explore these models in more detail.

3NF and Data Vault

The concept of the third normal form (3NF), as mentioned in [“Inmon Methodology”](#), is a data modeling technique that is often used in operational or transactional database normalization to reduce redundancy

and dependency. Some practitioners favor using the 3NF or other normalized forms for lakehouses because of the reduced data redundancy, adaptability, and improved consistency of data it offers.

Expanding on the principles of the 3NF, data vault introduces another normalized data modeling technique. It builds on the concepts of the 3NF by incorporating unique features such as hubs (unique business keys), links (data connections), and satellite tables (detailed descriptive information about the data).

The practical application of data vault modeling concepts generally takes place in the Silver layer of the Medallion architecture. This layer hosts both the raw data vault and the business vault. The raw vault presents structured, normalized representations of raw data, mapped to a conceptual business model. It integrates various sources using business keys, ensuring resilience against schema drift and tracking historical changes. The Silver layer also includes business vault elements like harmonization and intermediate transformations, which enrich the data and align it with enterprise-wide definitions. Point-in-time (PIT) and bridge tables further improve performance and querying, preparing data for use in the Gold layer. The roles of the Bronze and Gold layers, in this setup, remain largely unchanged, with the Bronze layer serving as the raw data staging area and the Gold layer transforming data for business intelligence and analytics.

The data vault structure is known for its adaptability, excelling in environments with frequent changes in data structures and business rules. This makes it particularly suitable for organizations with complex data needs. Several reasons might lead enterprises to prefer to use the 3NF or data vault structure in their Medallion architecture:

High integration needs

Enterprises with multiple, disparate source systems benefit from the 3NF and data vault's ability to create a unified, integrated, and consistent data model.

Complex enterprise environments

Organizations operating across multiple domains or with distributed teams require the flexibility and modularity offered by the 3NF and data vault.

Rapidly changing requirements and schema drift

The data vault's resilience to change and ability to adapt to evolving schemas make it ideal for dynamic environments where business and technical needs evolve frequently. By addressing these challenges, the data vault provides a future-proof framework for managing and delivering high-quality data.

Effective management of complex time dimensions

A data vault model can effectively manage multiple active timelines within the same records, such as creation time, functional processing time, and loading time. This capability enhances the ability to track and understand data evolution, which is crucial for audits, compliance, and detailed historical analysis.

Despite these advantages, using the 3NF or data vault for the Silver layer has drawbacks. While these modeling techniques offer improved flexibility and aim to save on storage costs, they are generally less favored in cloud-based data architectures due to scalability concerns. Practitioners often opt for wide, nested, denormalized tables because this setup maximizes cloud infrastructure efficiency. Denormalized tables avoid computationally expensive joins and data shuffling between Spark compute nodes, which can significantly slow down query performance.

Building a 3NF or data vault model can also be complex and usually requires more time and resources. Both models demand a detailed understanding of data relationships and dependencies, necessitating thorough planning and analysis. They also must adhere to stringent rules and structures in order to maintain their integrity and effectiveness. To address these challenges, organizations might consider frequently reviewing and implementing automation frameworks like [VaultSpeed](#).

Furthermore, the flexibility of lakehouse architecture causes practitioners to question the necessity of adopting a heavily normalized model like the data

vault, especially if significant schema changes are not a concern. The Medallion architecture simplifies data reloading from the Bronze layer through its queryable raw original tables, and Delta supports the time travel feature, enabling quick rollbacks of Silver-layer data to previous data versions.

TIP

For extensive insights on this topic, I highly recommend Simon Whiteley's [video on data vault offers](#).

Given the complexities and demands of models like the 3NF and data vault, organizations often choose larger denormalized tables for practical reasons. These tables offer better performance, ease of use, and simplify overall design, making them particularly appealing in environments where performance and simplicity are prioritized over strict data normalization. I'll come back to this subject when discussing the Gold layer.

In closing, data modeling is a complex topic that requires a strategic and nuanced approach tailored to specific business needs and context. It demands a careful balance of technical, business, and operational considerations. The approach to data modeling isn't simply black or white. Depending on your organization's specific requirements and constraints, you can select different data modeling strategies tailored to various use cases.

Moreover, deciding whether to integrate source systems in the Silver layer doesn't restrict you from applying some level of enterprise standardization to the data. Conforming to defined standards such as data types, centrally managed reference data, and naming conventions is encouraged. Some customers even implement basic business rules, like calculating new values. However, such activities should be kept to a minimum. The Silver layer should focus primarily on ensuring the data is clean and standardized rather than on heavily augmenting data and applying complex business rules. Let's address the need of data enrichments in detail by studying operational querying and machine learning.

Operational Querying and Machine Learning

Machine learning models perform best when the training data is closely aligned with the specific context or domain of the business problem being addressed. Therefore, many organizations utilize the Silver layer immediately for operational querying and machine learning workloads. However, this often leads to a debate on whether to enrich the data further, especially for machine learning applications. For example, transforming categorical data into a numerical format simplifies processing by machine learning algorithms. Many organizations, recognizing the need for an additional (sandbox or machine learning) layer for feature engineering and training models, go beyond the typical three layers in their data architecture.

These considerations play a crucial role in deciding where to position certain enrichment activities. If your goal is to enable operational reporting that necessitates data enrichment, I recommend beginning the enrichment process in the Silver layer. Although this approach may require extra adjustments during the merging process in the Gold stage, the increased flexibility is worth the effort.

Alternatively, if you value maintaining flexibility and prefer to separate concerns for easier management, consider delaying the enrichment of data until the Gold layer. This strategy isolates concerns and simplifies management, making it an effective approach for handling complex data structures.

Managing Overlapping Requirements

Let's continue discussing enrichments. There's an ongoing debate among engineers about where to apply business rules—should it be in the Silver or Gold layer? This discussion often revolves around themes of reusability and standardization. Generally, I suggest keeping transformations minimal in the Silver layer. The Gold layer, which is designed for end use, is where you should place business rules. This allows for customizations that meet specific use case needs and makes it easier to manage updates and maintenance.

However, this approach can lead to complications when the same integration needs to be reused often by other teams. If data in the Gold layer is made specific for one initial business unit, another team needing the same data might find themselves reconstructing the logic. This underscores the need for thoughtful planning in how the Silver and Gold layers are structured and where you choose to enrich and transform data. We'll explore the structuring again when discussing [curated and semantic layers](#) for the Gold layer.

Automation Tasks

The key to scalability lies in automating the majority of your tasks. However, it's crucial to recognize that not all processing steps across the various layers of the Medallion architecture can be easily standardized or automated to the same degree.

When it comes to automation, different tools and frameworks present unique considerations. As illustrated in [Figure 3-3](#), transitioning data from the source to the Bronze layer is particularly challenging. This complexity arises from the diversity of technologies and vendor solutions at the source system side. For instance, some vendors may use unique APIs or proprietary services for data extraction, which can result in a variety of data formats. For example, if a vendor supports only a CSV export format, then adapting your processes to accommodate this format becomes necessary.

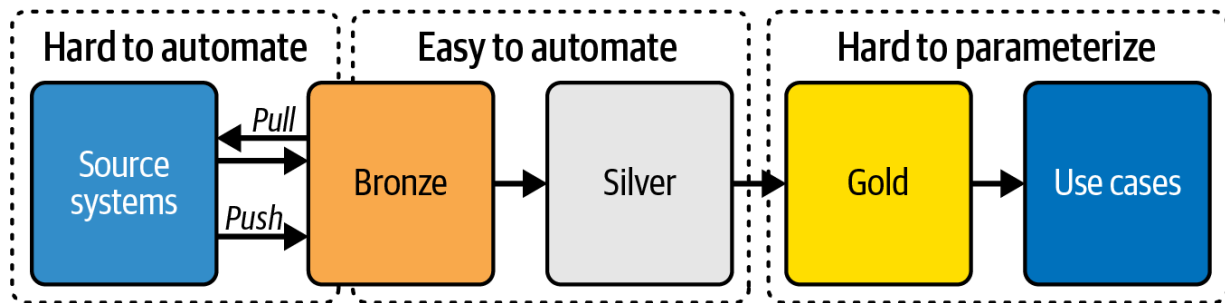


Figure 3-3. Highlighting the easily automated stages versus the complex processing steps within the Medallion architecture

Once you’ve overcome this initial hurdle and the data is available in a standardized (Delta Lake) format in the Bronze and Silver layers, you can proceed with further standardization and automation. At this stage, tools and metadata-driven frameworks become crucial, facilitating more streamlined and automated processing. This approach helps to maximize scalability and efficiency across the data processing architecture.

Returning to [Figure 3-3](#), the transition from the Bronze to the Silver layer generally appears to be more straightforward compared to the one from the source systems to the Bronze layer. At this stage, transformations are usually predictable and relatively simple, involving tasks such as renaming columns, applying filters, fixing data quality issues, utilizing lookup tables, and defaulting data. Due to its predictable nature, this phase of data engineering is easier to parameterize, making it more efficient and manageable.⁶ Consequently, many organizations rely on metadata-driven frameworks with common scripts and/or notebooks.

These frameworks enable you to define your data engineering tasks in a declarative manner. Essential elements like schema information, data quality rules, natural and business keys, and mapping rules are all stored within a metadata repository. This repository is then utilized to automatically generate the transformation code. By simply updating the metadata, you can effortlessly modify the transformations, which significantly automates the data engineering process.

Other frameworks to consider include dbt, the open source command-line tool previously mentioned, which excels by allowing transformations to be defined using templates, with a syntax similar to `SELECT` statements in SQL. Another declarative data engineering framework to consider, especially for those working within the Databricks environment, is Delta Live Tables (DLT). It not only facilitates transformations but also manages task orchestration, cluster management, monitoring, data quality, and error handling.

In [Figure 3-3](#), the final stage involves delivering refined data to consumers based on their unique requirements. This step is challenging to parameterize, primarily due to the complex business logic often required for integration, which isn't easily captured using metadata alone. However, by employing templates and services, you can introduce a level of standardization to your workflows. This approach streamlines the process, enhancing efficiency and manageability.

Understanding and managing the variances in different stages of data engineering is crucial for creating a scalable framework. By effectively leveraging automation tools and frameworks, organizations can optimize their data processing pipelines, ensuring they are both robust and adaptable.

The Silver Layer in Practice

In the Medallion architecture, data's journey began in the Bronze layer, where raw data from diverse sources was ingested and stored without alteration to preserve its original structure and integrity. This ensured a reliable dataset that could be referenced back to its source, which is crucial for both compliance and traceability.

Moving into the Silver layer, you cleanse, standardize, and (slightly) enrich the raw data. The primary focus is on cleaning and enhancing the representation of the dataset. This involves applying minimal transformations and augmentations, conducting data quality checks, and ensuring that the data is clean, usable, and in a standard format. The Silver layer also ensures that data is queryable and primed for further processing in the Gold layer.

TIP

Refine and transform source-aligned data in the Silver layer for operational consistency. This approach allows you to maintain a single source of truth, effectively replacing traditional operational data stores with a more dynamic and scalable lakehouse solution.

The structure of the Silver layer itself, whether it constitutes a single physical layer or includes multiple stages, largely depends on specific organizational requirements. For instance, to enhance auditability, you might divide this layer into three distinct stages: one for cleansing, another for conforming to standards, and potentially a third for building SCDs.

If you want to both align data ownership and integrate data, consider setting up separate layers: one for source system-aligned cleaned data and another for harmonized data. Whatever approach you take, keeping each stage clear and consistent is crucial.

Figure 3-4 represents how the Silver layer could look in practice.

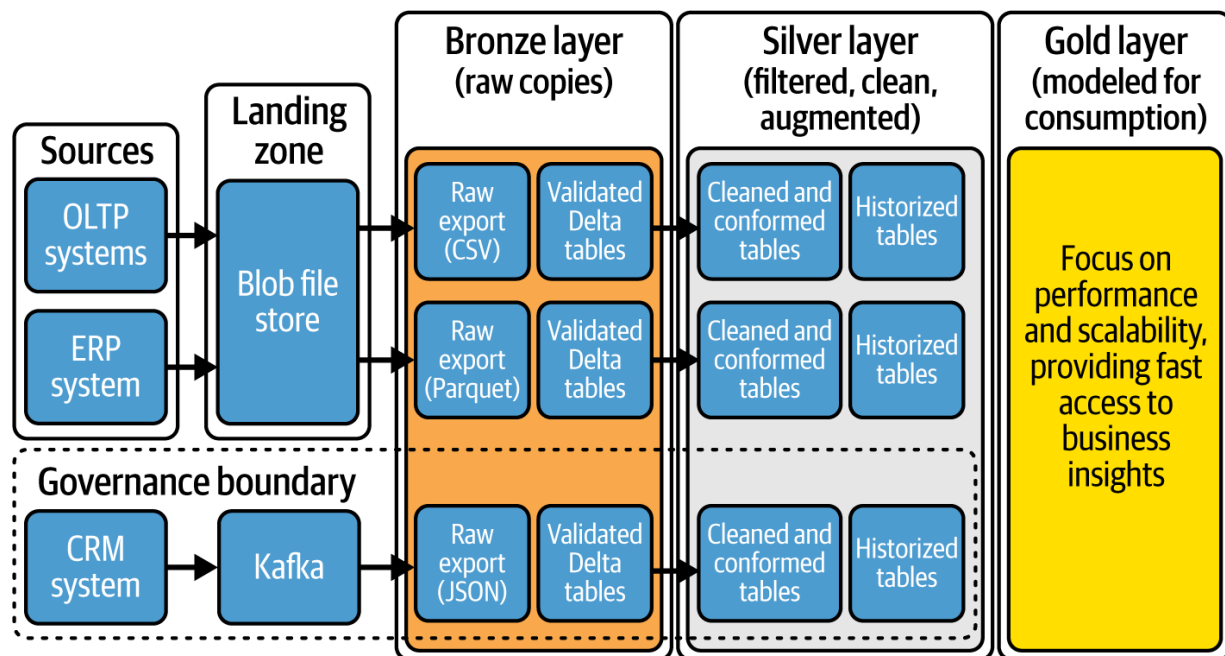


Figure 3-4. How the Silver layer could look in practice

The Silver layer typically mirrors the Bronze layer, presenting an exact representation but with crucial enhancements: tables are cleaned and data is standardized. This simple setup generally suits most organizational needs by ensuring data consistency and usability. However, the architecture of the Silver layer can be adjusted depending on the complexity of your data, the number of source systems involved, and the specific performance and flexibility requirements of your organization. This adaptability allows the Silver layer to not only address current data processing needs but also scale and evolve to meet future demands.

With data now cleaned and organized, the Gold layer takes on the role of refining this information to its most valuable form. This layer is where data is transformed into actionable insights that directly support decision-making processes. The operations in the Gold layer are complex because they involve advanced data modeling techniques, aggregation, and the application of business logic that aligns with strategic goals.

Gold Layer

Reaching the Gold layer, you encounter the most intricate part of your data architecture. This layer is the pinnacle of data refinement, designed specifically for decision making and reporting. As such, the data in the Gold layer is optimized for high-performance querying and analytics, ensuring it supports critical business functions effectively.

The complexity of the Gold layer cannot be overstated. It stems from the variety and intricacies of different source systems and the daunting task of merging all these into a single, harmonized view. This layer incorporates numerous complex business rules and engages in extensive post-processing activities such as calculations, enrichments, application-specific optimizations, corrections, transformations, aggregations, and many more.

Additionally, the work in the Gold layer is heavily influenced by business requirements, which can vary significantly. Some users may need simple flat structures, while others might require a more complex, well-modeled star schema that includes dimensions and facts. There may also be multiple user groups with potentially overlapping and contrasting requirements. This diversity of needs makes the Gold layer a challenging and complex part of the data architecture. Because the Gold layer is multifaceted, it's often broken down into multiple sublayers or stages to manage the complexity effectively.

Given this context, let's focus on the data model design by building a star schema for a straightforward Medallion architecture design. This approach and best practice will help you structure your data effectively, facilitating easier access and analysis. Once you have established a solid foundation with the star schema, you can explore the nuances of this design and further complexities that the Gold layer entails. This step-by-step approach ensures you address both the foundational and advanced aspects of data refinement in the Gold layer.

Star Schema

Star schemas will likely address most of your needs. They excel at conducting complex analyses on historical data and transforming data into entities such as an OLAP cube (a multidimensional data array based on online analytical processing). When you design star schemas, remember that their role extends beyond just boosting performance. They are shaped based on how users interact with data. Think of your data model as a public interface, similar to an API or a function. It's essential to make this interface intuitive and logically structured, just like any tool designed for user interaction. This way, users can navigate and utilize the data more effectively.

Clarity and relevance to business users are paramount. Even if a data model boasts exceptional performance, it will fall short if it doesn't align with how users conceptualize and navigate their domain. If business users find the model unnatural for slicing, dicing, and analyzing data according to their everyday business processes, the model will not be effective.

When designing a data model, prioritize a structure that reflects the end user's intuitive mental model: the star schemas. The star schema gets its name from its shape, a central fact table with dimension tables connecting to it. This approach ensures that the data model is not only efficient but also accessible and valuable to those who rely on it for making informed business decisions.

To develop a star schema, you must first understand the business requirements by engaging with stakeholders to capture their needs and expectations. Following this, you declare the granularity of the schema, which dictates the level of detail in the fact and dimension tables, ensuring they can be properly joined. The level of granularity also determines the aggregations that will be required.

STAR SCHEMA

In Kimball methodology, there are two key types of tables used to organize and manage data:

Fact table

A fact table is the central table in a star schema. A fact table stores quantitative data for analysis and is designed to be compact, fast, and adaptable.

Dimension table

Dimension tables are used to describe dimensions of the facts; they provide the context for the data. In essence, dimension tables store attributes related to the measurements in the fact tables, which helps in making the data understandable and readable.

In a typical star schema, a single fact table is surrounded by multiple dimension tables. The dimension tables are usually less voluminous than the fact table but have more text fields. This design allows for efficient storage and fast retrieval of data, which is vital for slicing-and-dicing operations in business intelligence and data analysis.

Next, you identify the dimensions that will structure the schema. For example, an air transportation company might need dimensions for time, locations, customers, airplanes, and so on.⁷ After establishing the

dimensions, pinpoint the facts that will populate the schema, completing the basic framework.

Loading the star schema involves two primary tasks: loading the dimension tables and loading the fact tables. These steps are crucial for operationalizing the schema to support business analysis and decision-making processes. Let's take a closer look at each of these tasks.

Loading the dimension tables

Loading dimension tables in a star schema is complicated by the need to handle SCDs, an incremental process that involves comparing incoming data with the existing data in the dimension table. This comparison helps identify new or changed data, manage surrogate keys, and appropriately insert or update dimension records.

In this step of processing SCDs, the ETL process scans the existing dimension table for the corresponding business key. If it finds a match, the process updates the existing record and inserts a new record with updated information. If it doesn't find a match, the process inserts a new record and assigns a new surrogate key. It's crucial to note that if business keys from different sources overlap, you must make adjustments. Using source system identifiers is one effective way to manage these overlaps.

It's essential to harmonize the data before inserting records into the dimension tables. This process involves transforming data into a common

format. Each source system is assessed to identify common attributes and values, a task that often requires significant business knowledge. During harmonization, records may need reorganization, cleaning, and correction. Codes might be decoded, multipart attributes could be split, and null field values might be replaced with mandatory values.

After harmonizing the records, they are ready for insertion into the dimension table. This process might involve several hops or stages. Typically, data is first loaded into a temporary table and then transferred into the dimension table. This multistage approach helps ensure data integrity and consistency in the dimension tables of the star schema.

Loading the fact tables

Loading the fact tables in a star schema, while seemingly simpler than loading dimensional tables, still presents its challenges. The primary task involves replacing the business keys, which describe business transactions, with surrogate keys linked to the dimension tables. Each row in the fact table includes foreign key references to rows in the dimension tables.

WARNING

It's crucial to create the dimension tables before the fact tables because fact tables rely on dimension tables for their surrogate keys. Without the presence of business keys in the dimension tables, it becomes impossible to locate and assign the appropriate surrogate keys.

During this loading process, you might encounter situations where certain relationships between the dimensions and facts in the tables cannot be established. This is what practitioners call an *early arriving fact*. In such cases, the creation of placeholder records becomes necessary. These records represent the missing entries and help maintain the integrity of the relationships between the dimension and fact tables, ensuring the star schema functions correctly.

Optimizing loads

When loading a star schema, various bottlenecks can occur, such as excessively long lookup times that force users to endure lengthy waits. To address this issue, you can optimize the data processing by incorporating administrative columns into your tables. For example, adding columns like `type1_hash` and `type2_hash` can streamline the detection of type 1 and type 2 changes during the ETL process.

NOTE

In a SCD1 model, you overwrite the old value with the new one, keeping no history. The SCD2 model preserves both current and historical records within the same file or table.

Additionally, including columns such as `creation_date` or `update_date` helps in identifying newly added data and modifications to existing data. These columns allow for a more efficient assessment of the

data that needs processing, thereby optimizing the loading process and minimizing bottlenecks. This strategic approach not only enhances performance but also improves the overall efficiency of managing and updating the star schema.

Star Schema Design Nuances

Managing the ETL process and building a star schema constitute a deep and complex process; the previous sections have merely introduced a few basic concepts. As you build your own star schema, you'll encounter a variety of nuances and design considerations inherent in this approach to data modeling. You'll likely discover that the process needs to be divided into several substeps, and that what initially appears as a single Gold layer will evolve and increase in complexity over time.

The Kimball approach to developing star schemas is essentially a set of design principles. It provides guidelines and common conventions for you or your developers to follow. However, there is significant diversity in how different organizations manage their models. Let's explore some alternative approaches to give you a broader perspective on the possibilities.

Curated, Semantic, and Platinum Layers

While implementing a star schema with facts and dimensions is a popular strategy for the Gold layer, they can become increasingly complex as

dimensional models expand and new data sources are incorporated. This growth often results in overlapping requirements that are similar but distinct enough to require different design approaches, potentially altering the design of the Gold layer and sometimes even the Silver layer.

For example, some organizations might add extra conformed or curation layers, with separate (semantic) layers for data marts that use star schemas. Occasionally, these additional layers are called *Platinum* layers, highlighting their specialized and highly refined nature. In such setups, it's common for organizations to follow some type of enterprise data modeling, like creating standard reference tables and conformed dimensions, which can be used across multiple data marts within a larger lakehouse architecture. Despite being considered complex and time-consuming, this approach remains popular because it ensures data reusability and standardization across different parts of the organization.

The approach of altering the design of the Gold layer comes with various nuances, influenced by factors like the size of the organization, the number of source systems intended for integration, semantic modeling requirements, and specific consumer requirements. Each of these factors can significantly impact how the data modeling strategy is implemented and sustained, making it crucial to tailor the approach to meet specific organizational needs and capacities. Now, let's delve into another design approach that simplifies the data model.

One-Big-Table Design

While designing star schemas can be highly beneficial, the process is also labor-intensive. Consequently, some practitioners prefer to implement a one-big-table (OBT) approach due to its speed and simplicity, which can outweigh considerations of model flexibility and extensibility.

OBT involves storing all relevant data for analysis or operations in a single, large table. This method avoids distributing data across multiple tables or organizing it according to more complex schemas, such as star or snowflake schemas. This approach is favored for several reasons:

Easier to manage

OBT is often simpler to manage and understand, particularly for those who are not specialists in data warehousing. This simplicity can be advantageous for smaller teams or projects where complexity can add significant overhead. Some practitioners point out that maintaining a single big table can reduce the overhead associated with managing multiple tables and relationships in a star schema. This can lead to easier data management and lower costs in terms of both time and resources.

Better performance

For certain types of queries, especially those that do not require aggregating large volumes of data from various dimensions, a single

big table can offer better performance. The elimination of joins that would be necessary in a star schema can lead to faster query execution times.

Flexible schema

A single big table provides a flexible schema that can be easier to modify and extend compared to a more rigid star schema. This can be particularly valuable in fast-paced environments where business requirements change frequently, necessitating quick adjustments to the data model.

Preferred by data scientists

Designs featuring one large table can also be preferred by data scientists who work with tools that expect data in one flat format. These OBT designs also offer convenience when transforming data into vector or graph-based datasets for modeling.

Great for long-term analysis

For datasets that inherently track changes over time (like sales or user activity data), a single, large table can make time-series analysis more straightforward. Analysts can observe historical trends and make future predictions based on a continuous stream of data.

Storing all of your data in a single table might seem to simplify the table structure, but it might make it more challenging to compose queries to

extract meaningful insights. Let's break this down with a concrete example.

Consider a table called `Orders` where, instead of distributing data across multiple relational tables, all data is stored in a single table using multiple columns such as `OrderID`, `OrderDate`, `CustomerID`, and `Products`. Within this table, the `Products` column stores a nested array of data, allowing the storage of one or multiple products associated with each order. Here's an example:

```
OrderID: 5687
OrderDate: 2024-11-15
CustomerID: 112233
Products: [
  {
    "ProductID": 101,
    "ProductName": "Apple iPhone 15 Pro",
    "Quantity": 2,
    "Price": 1299.99
  },
  {
    "ProductID": 205,
    "ProductName": "Dell XPS 15",
    "Quantity": 1,
    "Price": 1899.99
  }
]
```

Although all data is stored in a single table, the nested structure of the **Products** column can complicate queries that require aggregating or filtering data. For instance, aggregating data based on a specific product can be challenging. This is because the data is stored in a nested format, which requires additional processing to extract the relevant information.

Additionally, this strategy often leads to data duplication across multiple rows, which can increase memory demands for systems like Spark and, subsequently, degrade performance.

Managing changes within a single large table can also become complex quickly. For instance, adding a new field that impacts numerous rows can turn update operations into a significant endeavor. Typically, such changes necessitate recreating the entire table, which can be both time-consuming and resource-intensive. This highlights the challenges of maintaining a large, single-table database structure. While it offers simplicity in some areas, it requires careful consideration of potential complexities and performance issues that might arise, especially as data scales and evolves.

Serving Layer

So far, we have discussed the design and data modeling for a lakehouse, including the various layers involved. In the realm of technology architecture, especially concerning the Gold layer, you might still find it necessary to replicate data across other types of databases, in addition to lakehouses, using Delta tables. In this scenario, data is transferred from the

curated or presentation layer from the lakehouse to various other services like [Azure Data Explorer](#), [Azure SQL](#), or a graph database service, to name a few. This transfer makes it easier for end users to access the data and caters to the diverse needs of different business lines.

Take, for example, a business unit proficient in Azure SQL within an organization. This unit has developed applications and utilizes reporting tools that depend on data stored in Azure SQL databases. Rather than having the central team manage data in the lakehouse, this business unit manages the data within their own environment. They transfer data from the lakehouse to an Azure SQL database, essentially creating a “data mart.” This approach saves the business unit time in data preparation, enabling them to focus more on extracting deeper insights directly from the data. For more information, visit [James Serra’s blog post, “Serving Layers with a Data Lake”](#).

Something similar is often seen with reporting tools. For instance, Power BI, a widely used reporting tool, can directly connect to Delta tables in a lakehouse. Yet, many organizations opt for Power BI’s Import mode to ensure consistent performance and fine-grained security. In Import mode, Power BI replicates data from the lakehouse into its own in-memory engine, called VertiPaq.⁸ This approach enhances query performance and data retrieval efficiency, which is particularly beneficial for handling large datasets in a reporting model.

Lakehouse architectures often employ a diverse mix of technology services to satisfy various needs effectively. A typical lakehouse architecture includes serverless compute for ad hoc querying, Spark for big data processing, and Delta tables where the bulk of the data is stored. Relational databases might be used for handling more complex queries, time-series databases that cater to the Internet of Things (IoT) and streaming analysis, and reporting cubes like Power BI for facilitating analytics and visualization. The decision to complement the Gold layer with an additional layer using other database technologies often hinges on usability, compatibility with other services, flexibility, performance, and cost considerations.

This configuration of additional databases is a common setup for many organizations, demonstrating that lakehouse architecture is versatile but also can be tailored and integrated to specific requirements. It's crucial to understand that lakehouse architecture does not offer a one-size-fits-all solution but rather provides a flexible framework that can adapt to the diverse needs of different organizations.

The Gold Layer in Practice

The Gold layer of the Medallion architecture plays a crucial role in optimizing data for decision making and high-performance analytics. To achieve this, aligning closely with data governance is crucial to maintain compliance, integrity, and security. It's important to document and catalog

all datasets, maintain transparency about how data is used, and segment data for specific use cases. Clearly defining roles and responsibilities within this framework also ensures accountability and adherence to best practices. We'll revisit some of these concepts in [Chapter 11](#).

Furthermore, the data stored in the Gold layer needs a structure that is straightforward, self-explanatory, and optimized (see [“Managing Delta Tables”](#)) for reading. This setup must cater to various use cases that interact with this data. To design an effective model in this layer, it's crucial to align technical strategies with business needs. This eventually will result in additional physical (sub-)layers, as depicted in [Figure 3-5](#).

Organizations seeking a competitive edge in a data-driven landscape will benefit from the flexibility offered by the various modeling approaches. While it might be challenging to master the diverse formats on offer, your consumers will benefit from the ability to fully leverage their data assets offered by utilizing the most appropriate model.

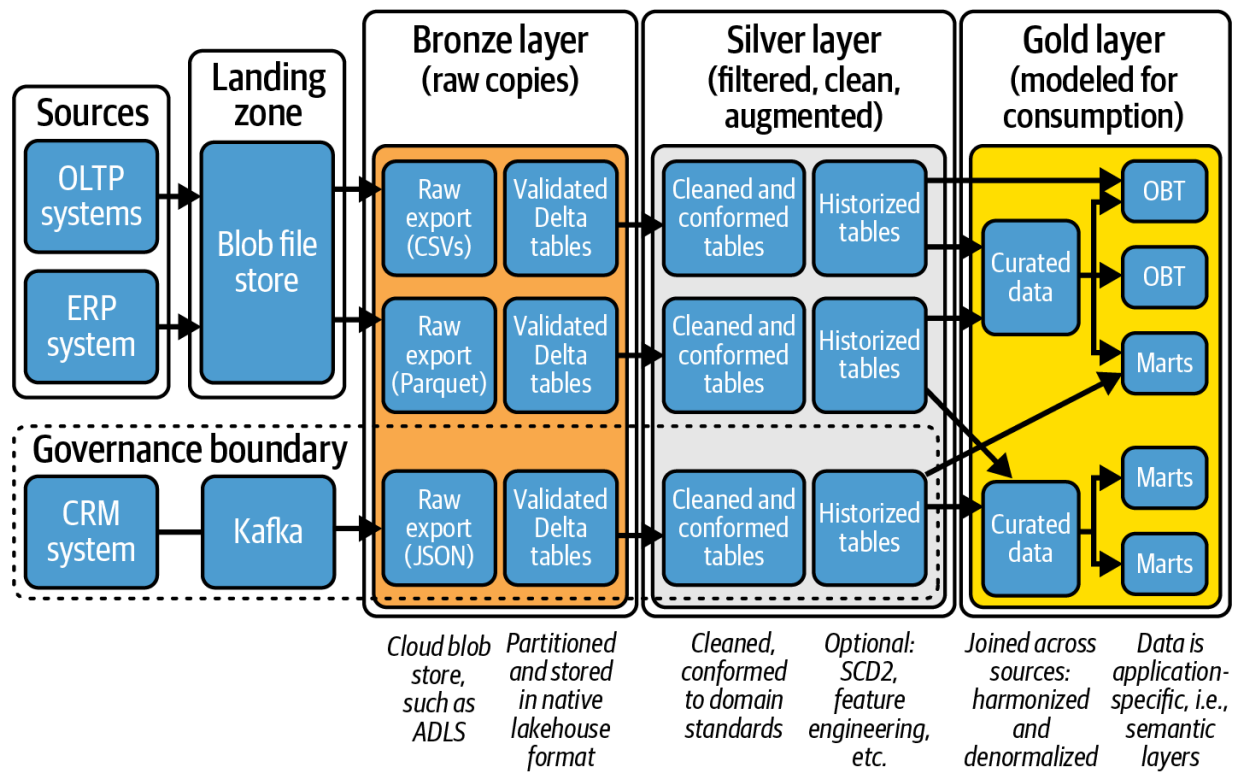


Figure 3-5. How the Gold layer could look in practice

Conclusion

The journey through the Medallion architecture and its layered approach—Bronze, Silver, and Gold—highlights that each layer of this architecture serves a unique purpose, tailored to refine data progressively from its raw form in the Bronze layer to a highly processed and decision-ready format in the Gold layer. [Table 3-1](#) provides a high-level overview of the layers and their key characteristics.

Table 3-1. Medallion layers overview table

Layer	Purpose	Data model	Applied Transformations	Fit for
Landing	Landing zone of raw data from source systems	Raw, as-is from source	None, as-is	Declarative for Consumption and
Bronze	Representation of validated raw data using standardized table formats	Source system schema	Minimal, such as applying filters and adding metadata	Not Ideal for Declarative Iceberg
Silver	Clean, historized, and read-optimized version, although still	Varies: mirrors the Bronze layer, subject-oriented, 3NF or data vault	Historized using SCD2, lightweight transformations, aligned with reference data,	Not Ideal for Declarative Iceberg

Layer	Purpose	Data model	Applied Transformations	Fit for
	source-oriented		feature engineering, etc.	
Gold	Optimized for value creation	Kimball data modeling or OBT	Harmonized, aggregated and with complex business logic applied	Not for Data Ice

But what makes the Medallion architecture so crucial for your organization’s data strategy? And what compelling conclusions can be drawn from this journey? The answer lies in its flexible, modular approach that allows organizations to tailor their data processes to specific needs. While the concept of three distinct layers offers a structured approach, it’s not a one-size-fits-all solution. The key is understanding the strengths and limitations of each layer, which can be adapted to better align with operational realities and strategic goals.

In order to take advantage of this flexible layering, it’s essential to recognize the importance of defining organization-wide standards. While flexibility is necessary, clear standards must guide engineering teams to

ensure consistency and effectiveness. Given the ambiguity in layer roles and the lack of clear industry definitions, defining what is expected from each layer, how data is validated and signed off at every stage, and the exact roles and responsibilities within the architecture are critical to maintaining a robust data strategy. We'll connect back to these topics in [Part IV](#).

Organizations that initially adopted meticulous models like the data vault sometimes faced performance issues, while those that prioritized performance with simplified models like one big table often encountered inflexibility. These experiences illustrate that effective data architecture requires a dynamic approach, balancing performance and flexibility. It is not just about choosing a model but about allowing for iterative refinements and adjustments to meet both current and future needs. To achieve this balance, organizations should consider setting clear standards for data modeling in relation to what is expected of each layer. By defining these expectations, organizations can ensure that their data is processed efficiently, but also adaptable and usable in relation to evolving business expectations.

Let's continue exploring the Medallion architecture by implementing a practical example, keeping in mind all the best practices and considerations we've learned so far. This hands-on experience will help us gain a deeper understanding of how to apply the Bronze, Silver, and Gold layers in a real-world scenario, providing valuable insights into data modeling and architecture complexities. In the next chapter, we will start by setting up the

necessary infrastructure for implementing the Medallion architecture using services such as Microsoft Fabric.

- Delta loads refer to the process of loading only the changes (newly added or modified data) since the last data load, rather than loading the entire dataset again.
- ! Depending on the compression algorithm, you can expect a size reduction of nearly 75% for your data in Parquet files from other formats, such as CSV.
- ! See [“Table Partitioning”](#) for more on partitioning.
- ! [TypeWidening](#) allows changes within the same type category. However, it doesn’t permit changes from one type category to another, like converting a string to a bool. To make such a change, you must perform a table-wide overwrite and set `overwriteSchema: true`.
- ! I discuss the differences between SCD1, SCD2, and SCD3 in [“Kimball Methodology”](#).
- ! *Parameterize* refers to the technique of defining and using parameters to control the processing steps of a data transformation pipeline. In this approach, parameters are variables or settings that can be adjusted externally without altering the underlying code of the scripts or notebooks.
- ! I’ll come back to this in [Part III](#) with more concrete examples.
- ! VertiPaq is an in-memory columnar data storage engine used by Microsoft Power BI, as well as other Microsoft products like Analysis Services and Excel’s Power Pivot.